

C++ 연습 문제: 11. 표준 라이브러리와 순차 컨테이너

1. C++ 표준 라이브러리에서 서로 다른 저장 방식을 가진 컨테이너들을 공통된 방식으로 순회하고 범위를 표현하기 위해 사용하는 개념은 무엇인가? (하)

1. 반복자(Iterator)
2. 스마트 포인터(Smart Pointer)
3. 가상 함수 테이블(Vtable)
4. 타입 특성(Type Traits)

2. 다음 중 메모리가 연속된 공간에 원소를 저장하여 인덱스를 통한 빠른 임의 접근(Random Access)을 지원하는 순차 컨테이너는 무엇인가? (하)

1. `std::list`
2. `std::forward_list`
3. `std::vector`
4. `std::map`

3. `std::vector`에서 원소를 추가할 때 현재 용량(capacity)을 초과하여 메모리 재할당이 일어날 때 발생하는 내부 동작으로 올바른 것은? (중)

1. 기존 메모리 공간의 크기만 그 즉시 늘어난다.
2. 새로운 더 큰 메모리 공간을 할당한 뒤, 기존 원소들을 새 공간으로 복사 또는 이동하고 기존 메모리를 해제한다.
3. 기존 원소를 모두 삭제한 뒤 새로운 원소만 새로 채운다.
4. 운영체제가 디스크에 임시 파일을 생성하여 부족한 공간을 대체한다.

4. `std::vector`의 `capacity()`와 `size()`에 대한 설명으로 올바른 것은? (중)

1. `size()`는 현재 할당된 총 메모리 용량을 뜻하고, `capacity()`는 들어 있는 원소의 개수를 뜻한다.
2. `capacity()`는 현재 할당된 총 메모리 용량을 뜻하고, `size()`는 실제 들어 있는 원소의 개수를 뜻한다.
3. `size()`와 `capacity()`는 항상 완전히 동일한 값을 가진다.
4. `capacity()`는 컨테이너가 가질 수 있는 최대 한계 크기를 뜻하며 변경할 수 없다.

5. `std::vector`에서 원소가 추가될 때 발생할 수 있는 불필요한 메모리 재할당 횟수를 미리 줄이기 위해, 저장할 공간의 크기를 미리 확보하는 함수는 무엇인가? (중)

1. `resize()`
2. `shrink_to_fit()`
3. `reserve()`
4. `clear()`

6. `std::vector`의 `reserve(10)`과 `resize(10)`의 차이점에 대한 설명으로 올바른 것은? (상)

1. `reserve()`는 컨테이너의 크기(size)를 10으로 늘리고, `resize()`는 용량(capacity)만 10으로 늘린다.

2. `reserve()`는 용량(capacity)을 10으로 늘리지만 원소를 생성하지 않으며, `resize()`는 크기(size)를 10으로 바꾸어 실제로 10개의 객체를 생성한다.
3. 두 함수는 완전히 동일한 동작을 수행한다.
4. `resize()`는 범위를 검사하지 않지만 `reserve()`는 범위를 검사한다.

7. `std::vector`에 원소를 삽입하거나 삭제할 때, 기존에 선언해둔 반복자, 포인터, 참조가 무효화(Invalidation)되는 주된 이유는 무엇인가? (상)

1. 컨테이너의 이름이 변경되기 때문이다.
2. 메모리 재할당으로 인해 주소가 바뀌거나, 삽입 위치 이후의 원소들이 메모리상에서 이동하기 때문이다.
3. 컴파일러가 최적화를 위해 임의로 반복자를 삭제하기 때문이다.
4. 정수형 인덱스 대신 반복자를 사용했기 때문이다.

8. `std::vector`와 달리 양방향 연결 리스트(Doubly Linked List) 구조로 이루어져 있어, 중간 삽입과 삭제가 원소 이동 없이 노드 연결 변경만으로 빠르게 처리되는 컨테이너는 무엇인가? (하)

1. `std::vector`
2. `std::deque`
3. `std::list`
4. `std::array`

9. `std::list`에 대한 설명으로 옳지 않은 것은? (중)

1. 메모리가 연속되어 있지 않아 인덱스를 통한 빠른 임의 접근(Random Access)을 지원하지 않는다.
2. 원소의 삽입과 삭제 시 기존 반복자나 포인터가 무효화되지 않는다.
3. 양방향으로 노드가 연결되어 있어 양쪽 끝뿐만 아니라 중간에서도 삽입/삭제가 효율적이다.
4. 데이터를 읽고 쓸 때 내부적으로 항상 $O(1)$ 의 임의 접근 성능을 보장한다.

10. 단방향 연결 리스트(Singly Linked List) 구조를 가지며, 메모리 오버헤드가 적고 뒤쪽에서만 노드가 연결된 순차 컨테이너는 무엇인가? (중)

1. `std::list`
2. `std::forward_list`
3. `std::deque`
4. `std::vector`

11. 양쪽 끝(앞과 뒤) 모두에서 원소의 삽입과 삭제를 효율적으로 처리할 수 있도록 설계된 자료 구조를 가진 순차 컨테이너는 무엇인가? (중)

1. `std::vector`
2. `std::deque`
3. `std::forward_list`
4. `std::set`

12. `std::vector`와 `std::deque`의 공통점과 차이점에 대한 설명으로 올바른 것은? (상)

1. 두 컨테이너 모두 임의 접근(Random Access)을 지원하지만, `std::vector`만 연속된 단일 메모리를 제공하여 `data()` 함수를 쓸 수 있다.
2. `std::deque`는 `std::vector`보다 앞쪽 삽입 연산이 훨씬 느리다.
3. `std::vector`는 양방향 큐 구조이고 `std::deque`는 단일 배열 구조이다.
4. 두 컨테이너 모두 원소를 추가할 때 메모리 재할당이 전혀 일어나지 않는다.

13. `std::list` 간에 노드 복사 없이 연결 고리(포인터)만 서로 통째로 옮겨 붙여서 성능을 극대화하는 멤버 함수는 무엇인가? (상)

1. `insert()`
2. `push_back()`
3. `splice()`
4. `merge()`

14. 다음 빈칸에 들어갈 알맞은 낱말은? (중) [C++ 표준 알고리즘(예: `std::sort`)은 특정 컨테이너 클래스의 구체적인 타입에 직접 의존하지 않고, 시작점과 끝점을 나타내는 () 범위를 받아 동작한다.]

1. 인덱스
2. 반복자(iterator)
3. 포인터
4. 레퍼런스

15. 다음 중 컨테이너의 원소를 수정하지 않고 읽기 전용으로 순회하며 처리할 때 사용하는 반복자 타입은 무엇인가? (하)

1. `const_iterator`
2. `reverse_iterator`
3. `mutable_iterator`
4. `volatile_iterator`

16. `std::vector`의 원소를 순회하며 특정 조건에 맞는 원소를 지우려 할 때, 반복자 무효화 문제를 피하기 위한 올바른 반복문 패턴으로 알맞은 것은? (상)

1. `for(auto it = v.begin(); it != v.end(); ++it) { v.erase(it); }`
2. `for(auto it = v.begin(); it != v.end(); ) { if (cond) it = v.erase(it); else ++it; }`
3. `for(int x : v) { v.erase(v.begin()); }`
4. `while(!v.empty()) { v.erase(v.begin()); }`

17. `std::vector`에서 원소들을 뒤로 밀어내는 비용을 줄이기 위해, 컨테이너의 용량을 현재 실제 크기(size)에 정확히 맞추어 남는 여유 공간을 없애는 멤버 함수는 무엇인가? (중)

1. `reserve()`
2. `resize()`
3. `shrink_to_fit()`
4. `clear()`

18. 컨테이너에 저장된 모든 원소를 제거하되, 할당되어 있던 메모리 용량(capacity)은 그대로 유지하고 싶을 때 사용하는 함수는 무엇인가? (하)

1. `clear()`
2. `shrink_to_fit()`
3. destructor 직접 호출
4. `erase()`

19. 다음 중 `std::deque`의 내부 저장 구조에 대한 설명으로 가장 올바른 것은? (중)

1. `std::vector`와 완전히 동일하게 하나의 거대한 연속된 메모리 블록만 사용한다.
2. 노드(조각) 단위로 메모리를 할당하고, 이를 관리하는 포인터 배열을 통해 가상의 연속된 임의 접근 공간을 제공한다.
3. 단방향으로만 연결된 노드들의 체인 형태로만 데이터를 유지한다.
4. 디스크 기반의 파일 시스템에 데이터를 직접 매핑하여 저장한다.

20. 표준 순차 컨테이너를 선택할 때 고려해야 할 설계 지침으로 가장 올바른 것은? (중)

1. 특별한 이유가 없다면 기본적으로 임의 접근이 빠르고 연속 메모리를 사용하는 `std::vector`를 우선 선택한다.
2. 모든 상황에서 성능이 무조건 동일하므로 아무 컨테이너나 골라 써도 무방하다.
3. 중간 삽입과 삭제가 빈번해도 무조건 `std::vector`를 사용하는 것이 성능상 가장 유리하다.
4. 크기가 항상 고정되어 있다면 `std::list`를 사용하는 것이 가장 표준적인 관행이다.